OceanJAX - Design and Implementation of a Differentiable Three-Dimensional Ocean Model Based on JAX

by
Zihan Yin
zy2770@columbia.edu

A paper submitted in fulfillment
of the requirements for the personal research course
in Department of Earth and Environmental Engineering
Course number EAEE 9305

at the
Columbia University

Date of complishment: 03/26/2026

OceanJAX - Design and Implementation of a Differentiable Three-Dimensional Ocean Model Based on JAX

Zihan Yin

Abstract

Accurate and efficient ocean modeling is crucial for understanding large-scale ocean circulation and its role within the climate system. However, many traditional ocean models were not designed to be fully differentiable, which limits their integration with modern machine learning methods, gradient-based parameter estimation, and data assimilation workflows. Consequently, this project utilizes JAX to develop a differentiable three-dimensional ocean modeling framework. The model employs a modular design, encompassing core components such as grid generation, spatial discretization operators, tracer transport, physical mixing, fluid dynamics, and time-stepping schemes. Throughout the research process, particular emphasis was placed on constructing physically consistent numerical operators while ensuring their compatibility with automatic differentiation mechanisms. Benefiting from the features of JAX, the framework natively supports hardware acceleration and lays the groundwork for future applications involving multi-GPU parallel computing and coupled machine learning models. The currently implemented model establishes a prototype for differentiable ocean simulation and paves the way for future functional extensions—including the adoption of variable-resolution grids and model initialization based on ocean reanalysis datasets such as ORAS5. This work provides an initial step toward combining physical ocean modeling with differentiable programming in a unified computational framework.

Contents

1 Introduction **1**

2 Model Framework **3**
2.1 Overall Architecture . . . 3
2.2 Computational Grid . . . 4
2.3 State Variables . . . 6
2.4 Physics Core and Governing Equations . . . 8
2.5 Module Organization . . . 10

3 Numerical Methods and Physical Parameterizations **13**
3.1 Discrete Operators and Spatial Discretization . . . 13
3.2 Time-Stepping Strategy . . . 15
3.3 Vertical Mixing and Richardson-Number-Based Closure . . . 17
3.4 Surface Forcing and Boundary Treatment . . . 19

4 JAX-Based Implementation **22**
4.1 Functional State Design and Solver Structure . . . 22
4.2 ORAS5 Data Interface and Regridding . . . 23
4.3 Runtime Workflow and NetCDF Output . . . 25
4.4 ML and Parallel Extensions . . . 27

5 Preliminary Verification and Discussion **29**
5.1 Numerical Consistency and Stability Checks . . . 29
5.1.1 Experiment 5.1.1: Resting Uniform Ocean Preservation Test . . . 29
5.1.2 Experiment 5.1.2: Stratified Rest Test over Real Topography . . . 30
5.1.3 Experiment 5.1.3: Zero-Side-Effect Verification of `NullClosure` . . . 32
5.1.4 Experiment 5.1.4: Time-Step Stability Scan under ORAS5-Initialized Forcing . . . 33
5.1.5 Experiment 5.1.5: Vertical Mixing Parameterization Test . . . 34
5.2 ORAS5-Initialized Integration Results . . . 35
5.2.1 Experiment 5.2.1: `oras5_cold` with Full Surface Forcing . . . 35
5.2.2 Experiment 5.2.2: `oras5_cold` without Surface Forcing . . . 36
5.2.3 Experiment 5.2.3: `oras5_full` without Surface Forcing . . . 36
5.2.4 Supplementary Experiment: 180-Day Atlantic Basin Integration . . . 38
5.3 Discussion of Current Model Behavior . . . 39

6 Limitations and Future Work **41**

7 Conclusion **42**

A Appendix **44**
.1 Default Model Parameters 44
.2 Source Code 45

Introduction

The ocean is a central component of the Earth system, regulating climate through the transport and redistribution of heat, freshwater, momentum, and tracers over a wide range of spatial and temporal scales. Numerical ocean models therefore play an essential role in understanding ocean circulation, climate variability, regional dynamics, and ocean–atmosphere interaction. By providing a physically consistent framework for the coupled evolution of velocity, temperature, salinity, and sea-surface height, such models have become indispensable tools in both fundamental research and practical prediction.

At the same time, the computational landscape of geophysical modeling is evolving rapidly. Many mature ocean circulation models were originally developed for large-scale forward simulation on conventional high-performance computing architectures. Although these models are scientifically powerful, they are not always designed for modern workflows that require native automatic differentiation, transparent modular structure, accelerator-oriented execution, or tight integration with machine learning components. As a result, there is increasing interest in ocean models that preserve essential physical fidelity while also serving as differentiable computational systems.

Differentiable scientific computing provides a particularly promising direction in this context. When numerical solvers are formulated as differentiable programs, gradients of model outputs with respect to parameters, initial conditions, and forcing fields can be obtained through automatic differentiation. This capability supports sensitivity analysis, parameter calibration, gradient-based optimization, and data assimilation, while also enabling hybrid physics–machine learning workflows. Modern array-programming frameworks such as JAX are especially attractive for this purpose because they combine composable functional programming with just-in-time compilation, automatic vectorization, and reverse-mode differentiation on modern accelerator hardware.

Motivated by these developments, this study presents OceanJAX, a modular and differentiable three-dimensional ocean modeling framework implemented in JAX. The current solver is based on the hydrostatic Boussinesq primitive equations with a prognostic free surface and a linear equation of state. It adopts a spherical Arakawa C-grid, supports variable horizontal resolution and custom bathymetry, and includes tracer transport, horizontal viscosity, implicit vertical viscosity and diffusion, and simplified Richardson-number-based vertical mixing. The framework also supports initialization and surface forcing from ORAS5 reanalysis products, thereby connecting the numerical core to realistic oceanic data.

A central design goal of OceanJAX is to combine physical transparency with computational extensibility. The model is organized around explicit state containers, reusable discrete operators, separated physics modules, and a time-stepping driver that remains compatible with just-in-time compilation and reverse-mode automatic differentiation. In its present form, the solver already supports differentiable forward integration through the

full trajectory, while preserving a clear separation between physical tendency evaluation, numerical integration, and data handling.

As the project has progressed, OceanJAX has also moved beyond a single-solver prototype. On the machine-learning side, the framework now includes a dedicated closure interface that provides a stable insertion point for learnable parameterizations. In the current implementation, ML corrections are injected centrally into the time-stepping loop after the explicit tracer tendencies are computed and before implicit vertical diffusion is applied. The first-version interface is intentionally conservative: it exposes only additional temperature and salinity tendencies together with a scaling factor for vertical diffusivity, while leaving momentum corrections for later work. This design preserves the integrity of the existing numerical core while creating a differentiable and testable pathway for future hybrid closures.

On the parallel-computing side, OceanJAX has completed Phase 1 of its multi-GPU roadmap. The current parallel layer distributes independent ensemble members across multiple devices by sharding the leading batch dimension, while replicating the grid and model parameters on each device. This strategy does not yet perform spatial domain decomposition of a single ocean simulation; instead, it provides an initial multi-accelerator capability aimed at ensemble execution, throughput scaling, and batched differentiable experimentation. Even so, it represents an important step toward a more scalable computational framework and establishes the software structure required for later phases of distributed execution.

The goal of this work is therefore not to compete immediately with mature production ocean models in terms of full physical complexity or operational readiness. Rather, the aim is to construct a research-oriented model core that retains essential ocean dynamics while remaining transparent, modular, differentiable, and extensible. In this sense, OceanJAX is intended as a bridge between physically based ocean modeling and emerging computational paradigms, including automatic differentiation, accelerator-aware parallel execution, and hybrid physics–machine learning modeling.

The remainder of this paper is organized as follows. Section 2 introduces the overall model framework, including the architecture, computational grid, state representation, governing equations, and module organization. Section 3 presents the numerical methods and physical parameterizations used in the current solver. Section 4 describes the JAX-based implementation, with emphasis on differentiable program structure, machine-learning interfaces, and the current parallel design. Section 5 presents preliminary verification results and discusses the present behavior of the model. Section 6 summarizes the current limitations of the framework and outlines future directions, and Section 7 concludes the paper.

Model Framework

2.1 Overall Architecture

OceanJAX is a modular, differentiable three-dimensional ocean modeling framework implemented in JAX. Its architecture is organized around a separation between grid construction, state representation, discrete numerical operators, physical process modules, time integration, and external data handling. This design improves readability and extensibility while remaining compatible with just-in-time compilation and reverse-mode automatic differentiation.

At the core of the framework, the computational domain is defined by an `OceanGrid` object. The grid module constructs an Arakawa C-grid in spherical longitude–latitude coordinates and stores the associated geometric information, including cell-center and face coordinates, horizontal and vertical spacings, metric factors, cell areas and volumes, Coriolis parameters, bathymetry, and wet–dry masks. This grid object provides the geometric foundation on which all subsequent numerical and physical calculations are performed.

The evolving ocean state is represented by the `OceanState` class, while scalar physical and numerical constants are collected in `ModelParams`. The state contains the three-dimensional velocity and tracer fields, the free-surface height, and several auxiliary history arrays required by the multi-step integration scheme. In particular, the framework stores previous momentum levels for leapfrog integration and previous tracer tendencies for Adams–Bashforth time stepping. This state-centered organization allows all major solver components to interact through a unified and explicit representation of the model variables.

On top of the grid and state layers, the `operators.py` module provides reusable finite-difference operators on the staggered grid. These include horizontal and vertical gradients, horizontal and vertical divergences, interpolations between staggered locations, and Laplacian-type operators. The operators are implemented as pure JAX functions and are written to remain compatible with JIT compilation and gradient propagation. As a result, the numerical discretization layer is cleanly separated from the physical parameterization layer.

The physical core is divided into three specialized modules. The `dynamics.py` module evaluates the momentum-related and diagnostic components of the hydrostatic Boussinesq system, including the linear equation of state, hydrostatic pressure anomaly, pressure-gradient force, Coriolis acceleration, free-surface tendency, and diagnosis of vertical velocity from continuity. The `tracers.py` module computes explicit tracer tendencies from advection and horizontal diffusion, and also provides the surface-flux tendencies for heat and freshwater forcing. The `mixing.py` module supplies implicit vertical diffusion and viscosity solvers, explicit horizontal viscosity, and a Richardson-number-based vertical diffusivity closure. This division of labor ensures that each module has a well-defined physical

role and avoids mixing time integration logic with process-level tendency calculations.

These modules are assembled by the `timeStepping.py` driver, which coordinates one complete model update. In the current implementation, a single time step first computes shared thermodynamic diagnostics, then advances horizontal momentum with a leapfrog scheme and Asselin–Robert filtering, applies implicit vertical viscosity, evaluates explicit tracer tendencies, advances tracers with an Adams–Bashforth scheme, applies implicit vertical diffusion, updates the free surface, diagnoses vertical velocity, and finally assembles the new masked state. For long integrations, the `run()` interface wraps the one-step solver inside `jax.lax.scan`, allowing the full trajectory to remain differentiable and JIT-compilable.

A separate data layer connects the numerical core to realistic ocean reanalysis products. The `oras5.py` module is designed in two stages: it first reads ORAS5 fields from NetCDF files and then regrids them onto the OceanJAX model grid. The loader supports both regular latitude–longitude products and native curvilinear NEMO/ORCA grids, and it can also read and interpolate surface forcing fields such as heat flux, freshwater flux, and wind stress. This design allows the same model framework to be used both for idealized experiments and for simulations initialized or forced by realistic data.

Overall, the architecture of OceanJAX is a numerical design strategy. By expressing the model as a composition of transparent grid-aware operators, process-based tendency modules, and differentiable time-stepping routines, the framework supports both conventional forward simulation and future extensions such as gradient-based parameter estimation, data assimilation, machine-learning parameterizations, and large-scale differentiable ocean modeling.

2.2 Computational Grid

The OceanJAX model adopts a three-dimensional Arakawa C-grid formulated in spherical longitude–latitude coordinates. The computational domain is defined on a structured rectangular patch of the sphere, with

$$N_x, \quad N_y, \quad N_z \tag{2.1}$$

denoting the numbers of grid cells in the zonal, meridional, and vertical directions, respectively. In the implementation, these dimensions are stored as static attributes of the grid object, while all geometric quantities are represented as JAX arrays.

Tracer variables are defined at cell centers,

$$(\lambda_i, \phi_j, z_k), \qquad i = 1, \ldots, N_x,\ j = 1, \ldots, N_y,\ k = 1, \ldots, N_z, \tag{2.2}$$

where λ and ϕ denote longitude and latitude, and z is positive downward. The zonal velocity component is located at east faces,

$$u_{i+\frac{1}{2},j,k}, \tag{2.3}$$

the meridional velocity component is located at north faces,

$$v_{i,j+\frac{1}{2},k}, \tag{2.4}$$

and the vertical velocity is defined at vertical interfaces,

$$w_{i,j,k+\frac{1}{2}}. \tag{2.5}$$

The free-surface height η is defined at the horizontal tracer locations,

$$\eta_{i,j}. \tag{2.6}$$

This staggered arrangement is advantageous because it places normal velocities directly on the faces through which fluxes pass, thereby improving the discrete representation of transport, pressure gradients, and continuity.

The horizontal grid is constructed from longitude and latitude bounds. For each horizontal direction, the model supports either uniform spacing or user-specified non-uniform spacing through normalized weighting arrays. The tracer-cell center coordinates are obtained from the midpoints of the horizontal cell edges, while the staggered face coordinates are taken directly from the corresponding east and north edges. As a result, the grid framework can represent variable horizontal resolution while preserving a structured memory layout.

Because the model is posed in spherical coordinates, the horizontal metric factors are spatially varying rather than globally constant. Let R denote the Earth's radius. Then the tracer-cell horizontal spacings are represented in the form

$$\Delta x^c_{i,j} = R\cos(\phi_j)\,\Delta\lambda_i, \qquad \Delta y^c_{i,j} = R\,\Delta\phi_j, \tag{2.7}$$

where $\Delta\lambda_i$ and $\Delta\phi_j$ are the local longitude and latitude increments in radians. At staggered velocity locations, the model stores separate metric arrays, including Δx^u, Δy^v, and Δx^v, so that the spherical geometry is treated consistently in the discrete operators. In particular, the north-face zonal width is evaluated using the latitude of the north face rather than the tracer-center latitude.

In the vertical direction, the grid is defined from a prescribed array of tracer-cell center depths,

$$z_k, \qquad k = 1, \ldots, N_z. \tag{2.8}$$

From these center depths, the model constructs the vertical interface depths $z_{k+\frac{1}{2}}$ by taking midpoints between adjacent centers, imposing

$$z_{\frac{1}{2}} = 0 \tag{2.9}$$

at the sea surface, and extrapolating the bottom interface one half-cell below the deepest tracer center. The tracer-cell thickness is then

$$\Delta z^c_k = z_{k+\frac{1}{2}} - z_{k-\frac{1}{2}}, \tag{2.10}$$

while an additional array of center-to-center distances is stored for vertical differencing on staggered locations. This construction provides a consistent geometric basis for hydrostatic pressure integration, vertical mixing, and vertical derivatives.

To reduce repeated geometric calculations during time stepping, the model precomputes cell areas and volumes. The horizontal tracer-cell area is

$$A_{i,j} = \Delta x^c_{i,j}\,\Delta y^c_{i,j}, \tag{2.11}$$

and the corresponding tracer-cell volume is

$$V_{i,j,k} = A_{i,j}\,\Delta z_k^c. \tag{2.12}$$

These geometric factors are reused throughout the discretization of fluxes, divergences, and source terms.

Bathymetry is represented through a two-dimensional water-column depth field

$$H_{i,j}. \tag{2.13}$$

If no bathymetry is provided, the model defaults to a flat-bottom domain whose depth equals the deepest vertical interface. Otherwise, the bathymetry array determines which cells and faces are wet or dry. The tracer-cell mask is defined as

$$m^c_{i,j,k} = \begin{cases} 1, & z_{k-\frac{1}{2}} < H_{i,j}, \\ 0, & \text{otherwise}, \end{cases} \tag{2.14}$$

so that a tracer cell is active whenever its top face lies below the local water depth.

Additional masks are defined at staggered locations. The zonal-velocity mask m^u is active only when the two tracer cells adjacent to a zonal face are both wet. The meridional-velocity mask m^v is defined analogously in the meridional direction. For vertical interfaces, the model uses two distinct masks. The first,

$$m^w_{i,j,k+\frac{1}{2}}, \tag{2.15}$$

is the dynamical vertical-face mask: it leaves the sea surface open wherever the surface tracer cell is wet, activates internal faces only when the two adjacent tracer cells are both wet, and closes the bottom face to enforce a no-normal-flow condition at the seafloor. The second,

$$m^{w,\mathrm{adv}}_{i,j,k+\frac{1}{2}}, \tag{2.16}$$

is used specifically for tracer advection and differs from m^w only at the sea surface, where it is forced to zero. This distinction ensures that kinematic vertical motion associated with the free surface can still be represented in the dynamical system, while advective tracer flux across the air–sea boundary is prohibited and must instead be handled through explicit surface forcing terms.

This masking strategy allows the model to represent irregular coastlines and bottom topography without changing the structured array layout of the solver. As a result, OceanJAX retains the implementation simplicity and computational efficiency of a structured grid while still accommodating realistic bathymetry and physically meaningful open and closed interfaces.

2.3 State Variables

The evolving ocean state in OceanJAX is represented by an `OceanState` object, while fixed physical and numerical constants are stored separately in a `ModelParams` object. This separation distinguishes time-dependent model fields from invariant coefficients and

provides a clean interface for functional time stepping, JIT compilation, and automatic differentiation.

The state contains the velocity components, tracer fields, and free-surface height defined on the staggered grid introduced in the previous subsection. Among these variables, u, v, T, S, and η are the principal prognostic fields advanced in time by the solver. By contrast, the vertical velocity w is treated as a diagnostic variable. It is not updated through an independent prognostic equation, but is recomputed from the discrete continuity relation so that the three-dimensional velocity field remains consistent with incompressibility and the free-surface kinematic condition.

In addition to the instantaneous physical fields, the state also stores the numerical history required by the time-integration scheme. For leapfrog stepping of momentum and free surface, the previous-time fields

$$u^{n-1}, \qquad v^{n-1}, \qquad \eta^{n-1} \tag{2.17}$$

are retained as `u_prev`, `v_prev`, and `eta_prev`. These arrays are used together with the current solution and the Asselin–Robert filter to advance the momentum and free-surface variables while suppressing the computational mode of the leapfrog method.

For tracer integration, the model uses an explicit Adams–Bashforth scheme. Accordingly, the state stores tracer tendencies from the previous one and two time levels,

$$G_T^{n-1}, \qquad G_S^{n-1}, \qquad G_T^{n-2}, \qquad G_S^{n-2}, \tag{2.18}$$

which are represented in the code as `T_tend_prev`, `S_tend_prev`, `T_tend_prev2`, and `S_tend_prev2`. These auxiliary arrays allow the solver to bootstrap from first-order forward Euler at the first step, switch to second-order Adams–Bashforth at the second step, and then use the full third-order Adams–Bashforth scheme thereafter.

The state also includes the scalar variables `time` and `step_count`. The former records the physical integration time, whereas the latter determines the startup logic of the multistep time-stepping scheme. Although they do not represent fluid fields directly, they form an essential part of the numerical state of the model.

The companion parameter container `ModelParams` stores the fixed coefficients required throughout the simulation. These include physical parameters such as gravitational acceleration g, reference density ρ_0, thermal expansion coefficient α_T, haline contraction coefficient β_S, reference temperature T_{ref}, and reference salinity S_{ref}. They also include numerical parameters such as horizontal and vertical viscosity, horizontal and vertical tracer diffusivity, time-step size, and filter coefficients.

The thermodynamic state is linked to density through a linear equation of state,

$$\rho_{i,j,k} = \rho_0 \left[1 - \alpha_T \left(T_{i,j,k} - T_{\mathrm{ref}}\right) + \beta_S \left(S_{i,j,k} - S_{\mathrm{ref}}\right)\right]. \tag{2.19}$$

However, density is not stored as an independent persistent state variable. Instead, it is diagnosed from temperature and salinity when needed for hydrostatic pressure and momentum calculations. Likewise, hydrostatic pressure anomaly is treated as a diagnostic quantity rather than a prognostic field.

To support different experimental settings, the framework provides multiple constructors for the state object, including zero-initialized states, resting states, and states assembled directly from externally supplied arrays. This flexible design makes the same state

representation suitable for debugging, idealized experiments, and simulations initialized from realistic data products.

Overall, the state representation in OceanJAX is more than a simple collection of fluid variables. It is a unified physical and numerical description of the evolving system, combining prognostic fields, diagnostic quantities required by the solver, multi-step history arrays, and scalar bookkeeping variables within a single differentiable data structure.

2.4 Physics Core and Governing Equations

This subsection focuses on the physical equations represented by the OceanJAX core and on how these equations are split across the dynamics, tracer, and mixing modules. Details of grid staggering and state storage have already been introduced in the previous subsections and are therefore not repeated here.

OceanJAX is based on the hydrostatic Boussinesq primitive-equation framework. Under the Boussinesq approximation, density variations are neglected everywhere except where they contribute to buoyancy and hydrostatic pressure gradients. The thermodynamic state is linked to density through a linear equation of state,

$$\rho = \rho_0 \left[1 - \alpha_T (T - T_{\mathrm{ref}}) + \beta_S (S - S_{\mathrm{ref}})\right]. \tag{2.20}$$

Rather than integrating the full hydrostatic pressure associated with ρ, the model uses the density anomaly

$$\rho' = \rho - \rho_0, \tag{2.21}$$

and constructs a baroclinic hydrostatic pressure anomaly p' from

$$\frac{\partial p'}{\partial z} = \rho' g, \tag{2.22}$$

with downward integration from the sea surface. This formulation avoids the spurious pressure-gradient error that would arise if the large background term $\rho_0 g z$ were discretized explicitly over stepped bathymetry.

The horizontal momentum equations are written in split form as

$$\frac{\partial u}{\partial t} = fv - g\frac{\partial \eta}{\partial x} - \frac{1}{\rho_0}\frac{\partial p'}{\partial x} + \mathcal{D}_u^{(h)} + \mathcal{D}_u^{(v)}, \tag{2.23}$$

$$\frac{\partial v}{\partial t} = -fu - g\frac{\partial \eta}{\partial y} - \frac{1}{\rho_0}\frac{\partial p'}{\partial y} + \mathcal{D}_v^{(h)} + \mathcal{D}_v^{(v)}, \tag{2.24}$$

where f is the Coriolis parameter, η is the free-surface height, and $\mathcal{D}^{(h)}$ and $\mathcal{D}^{(v)}$ denote horizontal and vertical viscous contributions. In the current implementation, the explicit momentum tendencies include only the Coriolis term, the barotropic and baroclinic pressure-gradient forces, and horizontal Laplacian viscosity. Vertical viscosity is handled separately through an implicit solver in the mixing module. Nonlinear momentum advection is intentionally not yet included, so the present system should be regarded as a hydrostatic primitive-equation core with linearized horizontal momentum forcing.

The free surface evolves from the depth-integrated continuity equation,

$$\frac{\partial \eta}{\partial t} = -\nabla_h \cdot \left(\int_{-H}^{0} \mathbf{u}_h \, dz \right), \tag{2.25}$$

where $\mathbf{u}_h = (u, v)$ is the horizontal velocity vector. Once the horizontal velocity has been updated, the vertical velocity is diagnosed from incompressibility,

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0. \tag{2.26}$$

In practice, w is obtained by downward integration of the discrete continuity equation rather than by solving an independent prognostic equation.

Tracer evolution is formulated for a generic scalar C, representing either temperature or salinity:

$$\frac{\partial C}{\partial t} = -\nabla_h \cdot (\mathbf{u}_h C) - \frac{\partial (wC)}{\partial z} + \nabla_h \cdot (\kappa_h \nabla_h C) + \frac{\partial}{\partial z} \left(\kappa_v \frac{\partial C}{\partial z} \right) + Q_C. \tag{2.27}$$

Here, horizontal advection is evaluated in flux form using a first-order upwind scheme in the production configuration, while horizontal diffusion is represented by a Laplacian operator with either scalar or spatially varying diffusivity. The vertical diffusive term is not treated explicitly; instead, it is passed to an implicit vertical solver in the mixing module.

A particularly important design choice is the treatment of the sea surface in the tracer equations. Advective tracer flux through the surface is prohibited by construction, and all air–sea tracer exchange is routed through explicit surface forcing tendencies. For temperature, the net surface heat flux Q_{heat} is converted into a top-layer tendency as

$$Q_T = \frac{Q_{\text{heat}}}{\rho_0 c_p \Delta z_1}, \tag{2.28}$$

where c_p is the specific heat capacity of seawater and Δz_1 is the thickness of the uppermost tracer layer. For salinity, the model uses a virtual salt-flux formulation driven by the net freshwater flux $E - P$,

$$Q_S = \frac{S_{\text{ref}}(E - P)}{\Delta z_1}. \tag{2.29}$$

This separation ensures that surface tracer forcing is not double-counted through both vertical advection and explicit boundary fluxes.

Vertical mixing is treated with an implicit diffusion operator solved column by column. For a tracer field ϕ, the discrete update takes the form

$$(I - \Delta t \, L_v) \, \phi^{n+1} = \phi^n + \Delta t \, R_\phi^{\text{exp}}, \tag{2.30}$$

where L_v is the vertical diffusion operator and R_ϕ^{exp} denotes the explicit tendency accumulated from advection, horizontal diffusion, and surface forcing. The resulting tridiagonal system is solved using a Thomas algorithm written in a differentiable JAX form. The same implicit strategy is also used for vertical viscosity acting on the horizontal velocity components.

The vertical diffusivity can be prescribed as a constant, but the present framework also includes a simplified Richardson-number-based closure. First, the gradient Richardson number is defined as

$$Ri = \frac{N^2}{S^2}, \tag{2.31}$$

where

$$N^2 = -\frac{g}{\rho_0}\frac{\partial \rho}{\partial z}, \qquad S^2 = \left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2. \tag{2.32}$$

The diffusivity is then enhanced under weakly stable shear and strongly increased under static instability. In the current implementation,

$$\kappa_v = \kappa_0 + \kappa_{\mathrm{conv}}\left(1 - \frac{Ri}{Ri_{\mathrm{crit}}}\right)^2, \qquad 0 \le Ri < Ri_{\mathrm{crit}}, \tag{2.33}$$

with an additional convective enhancement applied when $N^2 < 0$. This closure is deliberately simpler than a full K-profile parameterization, but it captures the essential idea that vertical mixing should strengthen in regions of shear instability or unstable stratification.

The physical tendencies described above are discretized through flux-form finite-difference operators supplied by `operators.py`. In particular, horizontal gradients, divergences, and Laplacian operators are evaluated with the spherical metric factors and wet-cell masks already defined by the grid module. As a result, the physical equations are expressed in a form that remains geometrically consistent on the structured spherical C-grid while preserving differentiability and modularity of the implementation.

Overall, the present physical core should be understood as a hydrostatic, differentiable regional ocean model that already includes free-surface dynamics, baroclinic pressure-gradient forcing, tracer transport, surface heat and freshwater forcing, and simplified turbulence closure, while still leaving several important extensions—most notably nonlinear momentum advection and a full KPP boundary-layer scheme—for future development.

2.5 Module Organization

The internal organization of OceanJAX follows a layered modular design in which geometry, state representation, numerical operators, physical process parameterizations, time integration, and external data handling are implemented as separate components. This separation is intended not merely for software readability, but also to enforce clear responsibility boundaries between mathematical representation, physical tendency evaluation, and numerical time stepping.

At the base of the framework is the grid module, `grid.py`. Its role is to define the computational geometry of the model domain, including the spherical Arakawa C-grid, metric factors, cell areas and volumes, Coriolis parameters, bathymetry, and wet–dry masks. Importantly, this module is purely geometric: it defines where quantities live and how the domain is shaped, but it does not evaluate tendencies or evolve the solution in time. This makes the grid object a reusable geometric foundation shared by every other component in the model.

The second layer is the state module, `state.py`, which provides the two central containers of the framework: `OceanState` and `ModelParams`. The former stores all evolving

fields and the auxiliary time-history arrays required by the multi-step solver, while the latter stores fixed physical and numerical coefficients. By packaging the full model state into a single differentiable object, this layer provides a consistent interface through which all physics and solver routines exchange information. At the same time, keeping parameters separate from the evolving state helps preserve conceptual clarity and supports future inverse or optimization workflows.

Above the grid and state layers sits the numerical-operator layer, implemented in `operators.py`. This module contains pure finite-difference and interpolation operators defined on the staggered grid, such as gradients, divergences, interpolations between staggered locations, and horizontal diffusion operators. A key design principle is that these functions are geometry-aware but physics-agnostic: they know about metric factors, masks, and boundary conditions, but they do not encode any specific governing equation. This allows the same discrete operators to be reused consistently by multiple physics modules.

The physical-process layer is divided into three separate modules. The `dynamics.py` module handles momentum-related and diagnostic calculations, including the equation of state, hydrostatic pressure anomaly, pressure-gradient forces, Coriolis terms, free-surface tendency, and diagnosis of vertical velocity. The `tracers.py` module handles explicit tracer tendencies, namely advection, horizontal diffusion, and surface tracer forcing. The `mixing.py` module handles subgrid transport closures and implicit vertical operators, including vertical diffusion, vertical viscosity, horizontal viscosity, and Richardson-number-based diffusivity parameterization. The purpose of this subdivision is to keep each physical component narrowly focused and to prevent tendency evaluation from being entangled with time-integration logic.

Time integration is then managed by `timeStepping.py`, which acts as the orchestration layer of the solver. Rather than recomputing physics internally in an opaque way, the time-stepping module explicitly calls the dynamics, tracer, and mixing modules in a prescribed sequence, applies the leapfrog, Asselin–Robert, and Adams–Bashforth schemes where appropriate, and assembles the updated state. In this sense, `timeStepping.py` serves as the numerical controller of the model rather than as a repository of physical formulations. The distinction is important because it ensures that the physical modules remain reusable and independently testable.

A separate data-interface layer is implemented in `oras5.py`. Its purpose is to isolate external data access from the numerical core. This module reads ORAS5 reanalysis files, resolves variable-name aliases, supports both regular and curvilinear source grids, performs interpolation onto the OceanJAX grid, and constructs masked initial states or surface forcing fields. By confining all reanalysis-specific logic to this module, the solver core remains independent of the details of a particular dataset. This makes it easier to substitute other reanalysis products or observational datasets in future work without modifying the numerical model itself.

Taken together, these modules form a one-directional dependency structure. The grid and state layers provide shared foundations; the operators depend on the grid; the physics modules depend on the grid, operators, and state; the time stepper depends on all solver-side modules; and the ORAS5 interface provides external inputs to the state without altering the internal solver contracts. Such a structure reduces coupling between components, improves testability, and helps ensure that modifications in one part of the code do

not propagate unpredictably through the rest of the model.

This organization is especially well suited to a differentiable JAX-based modeling framework. Because each module is written as a transparent transformation on arrays or structured state objects, the full model remains compatible with just-in-time compilation, vectorization, and automatic differentiation. More importantly, the modular interfaces already create natural insertion points for future extensions, such as nonlinear momentum advection, full K-profile parameterization, machine-learning closures, and multi-device parallel execution. Therefore, the current module organization should be understood not only as an implementation choice, but as part of the research design of OceanJAX itself.

Numerical Methods and Physical Parameterizations

3.1 Discrete Operators and Spatial Discretization

This subsection describes the spatial discretization strategy used in OceanJAX and the discrete operators that support the physical tendency calculations. The discussion here focuses on how continuous differential terms are represented numerically, rather than on the grid geometry itself, which has already been introduced in the previous chapter.

OceanJAX adopts a finite-difference formulation on a structured spherical Arakawa C-grid, but many of the core operators are written in flux form rather than as direct pointwise second derivatives. This choice is especially important for tracer transport and continuity because it allows the numerical scheme to remain closely tied to discrete conservation principles. In the implementation, all operators are written as pure JAX functions, with geometry and masking supplied explicitly through the grid object. As a result, the discretization layer remains reusable, differentiable, and cleanly separated from the higher-level physics modules.

The basic first-order derivative operators are the zonal, meridional, and vertical gradients, denoted here schematically as

$$G_x(\phi), \qquad G_y(\phi), \qquad G_z(\phi). \tag{3.1}$$

For a tracer field ϕ, the horizontal gradients are evaluated as one-sided differences from cell centers to staggered velocity faces:

$$G_x(\phi)_{i+\frac{1}{2},j,k} = \frac{\phi_{i+1,j,k} - \phi_{i,j,k}}{\Delta x^u_{i,j}}, \tag{3.2}$$

$$G_y(\phi)_{i,j+\frac{1}{2},k} = \frac{\phi_{i,j+1,k} - \phi_{i,j,k}}{\Delta y^v_{i,j}}. \tag{3.3}$$

The zonal direction is treated as periodic, while the meridional direction uses a zero-gradient condition at the northern boundary. In both cases, the resulting face values are multiplied by the corresponding wet-face masks so that no gradient or flux crosses dry points or land boundaries.

The vertical gradient is defined on w-faces:

$$G_z(\phi)_{i,j,k+\frac{1}{2}} = \frac{\phi_{i,j,k+1} - \phi_{i,j,k}}{\Delta z^w_{k+\frac{1}{2}}}, \tag{3.4}$$

for interior faces, while the surface and bottom faces are explicitly set to zero inside the operator. This treatment enforces zero diffusive flux at the top and bottom within the

operator layer itself, leaving the dynamical surface kinematic condition to be imposed separately at the time-stepping level.

In addition to gradients, the model relies heavily on discrete divergence operators. The horizontal divergence is written in flux form at tracer points:

$$\nabla_h \cdot \mathbf{F} = \frac{F^u_{i+\frac{1}{2},j,k} - F^u_{i-\frac{1}{2},j,k} + F^v_{i,j+\frac{1}{2},k} - F^v_{i,j-\frac{1}{2},k}}{A_{i,j}}, \tag{3.5}$$

where the face fluxes already include the appropriate metric widths and wet-face masks. In practice, the zonal flux uses the meridional tracer-cell width Δy^c, while the meridional flux uses the zonal north-face width Δx^v, thereby preserving the correct spherical metric dependence. The vertical divergence is represented as

$$\frac{\partial w}{\partial z} \approx \frac{w_{i,j,k+\frac{1}{2}} - w_{i,j,k-\frac{1}{2}}}{\Delta z^c_k}, \tag{3.6}$$

with the vertical face values first gated by the w-face mask. This divergence operator is used directly in continuity-related diagnostics and in the vertical part of flux-form tracer transport.

A second important component of the discretization layer is the collection of interpolation operators between staggered locations. These include mappings from tracer points to velocity faces, from velocity faces back to tracer centers, and between tracer centers and vertical faces. In the present implementation, these interpolations are arithmetic averages of the adjacent valid points, with boundary behavior handled consistently with the masking rules. These interpolants are used repeatedly throughout the model, for example when evaluating diffusive coefficients at faces or when converting between native variable locations in the physics modules.

Horizontal diffusion is represented by a flux-form Laplacian operator. For a scalar field ϕ, the horizontal diffusive term is written as

$$\nabla_h \cdot (\kappa_h \nabla_h \phi) = \frac{F^x_e - F^x_w + F^y_n - F^y_s}{A}, \tag{3.7}$$

where each face flux is computed from a discrete gradient multiplied by the corresponding metric width and horizontal diffusivity. When κ_h is spatially varying, it is first interpolated from cell centers to faces before the fluxes are formed. This design is used in the tracer module so that horizontal diffusion can be treated consistently for both constant and space-dependent diffusivity fields.

The operator layer also provides a purely geometric horizontal Laplacian

$$\nabla^2_h \phi, \tag{3.8}$$

and its repeated application,

$$\nabla^4_h \phi = \nabla^2_h \left(\nabla^2_h \phi \right), \tag{3.9}$$

as a biharmonic operator. The Laplacian is defined in flux form and respects the same metric and masking structure as the other operators. The biharmonic form is included as a scale-selective dissipation option that can be useful at higher resolution, although it is not yet the default tracer formulation in the present solver.

For horizontal momentum diffusion, the implementation does not directly reuse the tracer-cell Laplacian. Instead, the mixing module defines separate scalar Laplacian operators at u- and v-points, using the geometry of the velocity control volumes themselves. This distinction is necessary because the spacing, face areas, and active masks at velocity locations differ from those at tracer centers. The resulting viscosity tendencies therefore remain consistent with the staggered velocity discretization.

An important unifying principle across all spatial operators is the consistent use of masks. Every face flux is multiplied by the appropriate face mask before differencing, and every tracer-centered output is multiplied by the tracer-cell mask. This means that coastlines, seafloor boundaries, and dry cells are handled algebraically within the operator definitions rather than through special-case branching logic. Such a design is particularly advantageous in JAX because it preserves array-level transparency, avoids data-dependent control flow, and remains fully compatible with just-in-time compilation and reverse-mode automatic differentiation.

These discrete operators form the numerical backbone of the OceanJAX solver. The dynamics module uses them to evaluate pressure gradients, horizontal divergences, and continuity-based diagnostics; the tracer module uses them to construct advective and diffusive flux divergences; and the mixing module uses closely related metric-aware Laplacian forms for momentum viscosity. In this way, the spatial discretization layer provides a common numerical language through which the different physical processes are expressed consistently on the spherical staggered grid.

3.2 Time-Stepping Strategy

The OceanJAX solver advances the model state using a split time-integration strategy that distinguishes between explicit dynamical tendencies, explicit tracer tendencies, and vertically stiff diffusive processes. The purpose of this design is to retain a relatively simple and transparent explicit treatment for horizontal dynamics and transport, while handling the restrictive vertical mixing terms implicitly in order to avoid severe stability constraints on the time step.

At the highest level, one model step is carried out by the function `step()`, which orchestrates the physics modules in a fixed sequence. Shared thermodynamic diagnostics are computed once at the beginning of each step and then reused by the momentum solver. The explicit horizontal momentum tendencies are advanced first, followed by implicit vertical viscosity. The tracer equations are then advanced using explicit advection–diffusion tendencies together with surface forcing, after which implicit vertical diffusion is applied. Finally, the free surface is updated and the vertical velocity is diagnosed from continuity. This ordering ensures that each process enters the update exactly once and that explicit and implicit contributions are cleanly separated.

For horizontal momentum, OceanJAX adopts a leapfrog scheme. If G_u^n and G_v^n denote the explicit momentum tendencies at time level n, the update is written as

$$u^{n+1} = u^{n-1} + 2\Delta t\, G_u^n, \qquad v^{n+1} = v^{n-1} + 2\Delta t\, G_v^n. \tag{3.10}$$

Because leapfrog requires a previous time level, the first step is bootstrapped with a forward-Euler start,

$$u^1 = u^0 + \Delta t\, G_u^0, \qquad v^1 = v^0 + \Delta t\, G_v^0, \tag{3.11}$$

which is implemented by replacing $2\Delta t$ with Δt when `step_count = 0`. To suppress the computational mode inherent to leapfrog integration, the model applies an Asselin–Robert filter to the current-time momentum fields:

$$u^n_{\text{filt}} = u^n + \alpha \left(u^{n+1} - 2u^n + u^{n-1} \right), \tag{3.12}$$

$$v^n_{\text{filt}} = v^n + \alpha \left(v^{n+1} - 2v^n + v^{n-1} \right), \tag{3.13}$$

where α is the Asselin filter coefficient. The filtered fields are then stored as the previous-time momentum level for the next step.

After the explicit leapfrog advance, vertical viscosity is applied implicitly to the horizontal velocity components. In other words, the leapfrog result is not yet the final updated momentum field; instead, it is passed through the implicit vertical solver from the mixing module. This operator-splitting approach allows the model to retain the simplicity of explicit horizontal momentum forcing while avoiding the strong time-step restriction that explicit vertical viscosity would impose in thin upper layers.

Tracer evolution is treated differently from momentum. The explicit part of the tracer tendency, including advection, horizontal diffusion, and optional surface forcing, is advanced using an Adams–Bashforth multi-step scheme. Let G^n_C denote the explicit tendency for a tracer C. Then the general third-order update is

$$C^{n+1} = C^n + \Delta t \left(a_0 G^n_C + a_1 G^{n-1}_C + a_2 G^{n-2}_C \right), \tag{3.14}$$

with the standard AB3 coefficients

$$(a_0, a_1, a_2) = \left(\frac{23}{12}, -\frac{16}{12}, \frac{5}{12} \right). \tag{3.15}$$

Because the scheme requires tendency history, the first two steps are bootstrapped explicitly through `step_count`: the first step uses AB1 (forward Euler), the second uses AB2, and only from the third step onward does the solver use the full AB3 coefficients. This explicit bootstrap logic is numerically safer and clearer than relying on zero-filled tendency history alone.

An important implementation detail is that the tracer advection step uses the Asselin-filtered horizontal velocity fields rather than the raw leapfrog-updated velocities. This choice ensures that the tracer solver sees a dynamically consistent velocity state, rather than one contaminated by the leapfrog computational mode. At the same time, tracer advection still uses the vertical velocity from the previous state, because the new w field is not diagnosed until the end of the time step after the free-surface update.

Following the explicit Adams–Bashforth tracer advance, vertical diffusion is applied implicitly:

$$(I - \Delta t\, L_v)\, C^{n+1} = C^*, \tag{3.16}$$

where C^* denotes the explicit tracer update and L_v is the vertical diffusion operator. This implicit step is solved column by column using the Thomas algorithm. The same strategy is used for temperature and salinity, ensuring numerical stability even when the vertical grid is fine or when the effective vertical diffusivity becomes large.

The current framework also includes a machine-learning closure hook inside the time-stepping loop. When a closure is provided, it is evaluated after the explicit tracer tendencies have been computed and before implicit vertical diffusion is applied. In the present

version, the closure can add corrections to the temperature and salinity tendencies and can scale the background vertical diffusivity. If no closure is provided, this stage is skipped entirely and the numerical path remains identical to the pure-physics solver. This design makes the insertion point for learnable parameterizations explicit, localized, and compatible with automatic differentiation.

The sea-surface height is advanced consistently with the leapfrog treatment of momentum. If G_η^n denotes the free-surface tendency from the depth-integrated continuity equation, the update is written as

$$\eta^{n+1} = \eta^{n-1} + 2\Delta t\, G_\eta^n, \tag{3.17}$$

with a forward-Euler bootstrap at the first step, analogous to the momentum update. An Asselin–Robert filter is then applied to the current free-surface field in the same way as for u and v. After the free surface has been updated, the vertical velocity is diagnosed from the discrete continuity equation, and the surface vertical face is assigned the kinematic value

$$w_{k=0} = \frac{\partial \eta}{\partial t}. \tag{3.18}$$

This ordering ensures consistency between the interior incompressibility constraint and the free-surface boundary condition.

The full `step()` routine may therefore be summarized as an 11-stage update: shared diagnostics, explicit momentum tendencies, leapfrog momentum advance, Asselin filtering, implicit vertical viscosity, explicit tracer tendencies, optional ML closure correction, Adams–Bashforth tracer advance, implicit vertical diffusion, free-surface leapfrog update, and continuity-based diagnosis of vertical velocity. This sequence is not arbitrary; it reflects the numerical splitting strategy of the model and the responsibility boundaries between the dynamics, tracers, mixing, and closure modules.

For multi-step simulations, the solver uses a `run()` wrapper built on `jax.lax.scan`. This design avoids Python-side iteration, allows the full trajectory to remain JIT-compilable, and preserves compatibility with reverse-mode automatic differentiation through the time-integration loop. Time-varying surface forcing can be supplied as a sequence with a leading time dimension, while the same stepping logic is reused unchanged at every iteration.

Overall, the time-stepping design of OceanJAX combines a conventional geophysical split-explicit philosophy with implementation choices tailored to a differentiable JAX framework. Leapfrog and Adams–Bashforth provide efficient explicit treatment of momentum and tracer transport, implicit vertical operators stabilize the stiff mixing processes, and the centralized stepping routine creates a natural interface for both machine-learning closures and future parallel extensions.

3.3 Vertical Mixing and Richardson-Number-Based Closure

Vertical transport processes in ocean models are numerically stiff because the vertical grid spacing is often much smaller than the horizontal spacing, especially near the surface. As a result, an explicit treatment of vertical diffusion or vertical viscosity would impose a severe stability restriction on the time step. For this reason, OceanJAX treats vertical tracer diffusion and vertical momentum viscosity separately from the explicit horizontal dynamics and transport terms, and solves both processes implicitly in the vertical direction.

For a generic tracer field ϕ, the vertical diffusion problem is written in operator form as

$$(I - \Delta t\, L_v)\, \phi^{n+1} = \phi^n + \Delta t\, R_\phi^{\exp}, \tag{3.19}$$

where L_v is the vertical diffusion operator and $R_\phi^{\exp}$ denotes the explicit tendency accumulated from advection, horizontal diffusion, and surface forcing. In discrete flux form, the vertical diffusive flux through a vertical face is

$$F_{k+\frac{1}{2}} = -\kappa_{k+\frac{1}{2}} \frac{\phi_{k+1} - \phi_k}{\Delta z^w_{k+\frac{1}{2}}}, \tag{3.20}$$

and the resulting tracer tendency within layer k is obtained from the divergence of the top and bottom fluxes divided by the tracer-layer thickness Δz^c_k. This leads to a tridiagonal linear system in the vertical direction for each water column.

OceanJAX solves this system column by column using a differentiable Thomas algorithm implemented with `jax.lax.scan`. The tridiagonal coefficients are assembled from the vertical diffusivity, the cell thicknesses, and the active vertical-face mask. In this way, the entire implicit solve remains compatible with JIT compilation and reverse-mode automatic differentiation. The same algorithmic structure is reused throughout the model, which keeps the implementation compact while preserving numerical clarity.

A subtle but important aspect of the tracer mixing operator is its treatment of the sea surface. The tracer solver uses a dedicated vertical advection/diffusion mask in which the surface face is closed. This means that the implicit vertical diffusion operator enforces zero diffusive tracer flux through the air–sea interface. The motivation is physical as well as numerical: surface exchange of temperature and salinity is handled explicitly through the surface forcing terms, so allowing an additional diffusive surface flux inside the implicit operator would lead to double counting of air–sea exchange. The same no-flux condition is applied at the bottom boundary.

Vertical viscosity for the horizontal velocity components is treated analogously, but with an important distinction in masking. Because the u and v velocity components live on different staggered control volumes from tracers, their active vertical connections cannot be inferred directly from the tracer vertical mask. Instead, OceanJAX builds velocity-consistent vertical face masks separately for u and v: an internal face is open only if the velocity point is wet in both adjacent layers. This avoids spurious coupling across topographic steps or seamount edges, where tracer cells may still exist nearby but a given velocity column may already be dry. The implicit vertical viscosity update for velocity therefore takes the same algebraic form as the tracer diffusion solve, but with viscosity coefficients and velocity-specific masks.

In addition to the implicit vertical operators, the framework includes a physically motivated closure for the vertical diffusivity based on the gradient Richardson number. The diagnostic Richardson number is defined as

$$Ri = \frac{N^2}{S^2}, \tag{3.21}$$

where the squared buoyancy frequency is

$$N^2 = -\frac{g}{\rho_0} \frac{\partial \rho}{\partial z}, \tag{3.22}$$

and the vertical shear magnitude is

$$S^2 = \left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2 . \tag{3.23}$$

Both quantities are evaluated at vertical faces using centered finite differences. A small floor is applied to S^2 in order to avoid division by zero, but the diagnostic Richardson number itself is not clipped in the diagnostic routine, so that negative values associated with static instability are preserved.

Based on this diagnostic field, OceanJAX implements a simplified Richardson-number-dependent vertical diffusivity:

$$\kappa_v = \kappa_0 + \kappa_{\mathrm{conv}} \left(1 - \frac{Ri}{Ri_{\mathrm{crit}}}\right)^2, \qquad 0 \le Ri < Ri_{\mathrm{crit}}, \tag{3.24}$$

together with an additional convective enhancement when

$$N^2 < 0. \tag{3.25}$$

In practice, the closure first clamps Ri to the interval $[0, Ri_{\mathrm{crit}}]$ for the shear-mixing branch, while still using the signed buoyancy frequency to detect statically unstable layers. The resulting diffusivity therefore increases both under weakly stable shear and under convective overturning.

This closure should be interpreted as a simplified stability-dependent background mixing parameterization rather than as a full K-profile parameterization. It does not include explicit boundary-layer depth diagnosis, nonlocal transport, or counter-gradient terms. Its purpose is instead to provide a lightweight and physically interpretable mixing formulation that is suitable for a differentiable research framework and can later be replaced or augmented by more sophisticated closures, including machine-learning-based parameterizations.

An important implementation note is that the Richardson-number-based diffusivity has already been implemented as a diagnostic closure in the framework, but it is not yet the default diffusivity used in the main production time-stepping path. In the current solver configuration, the implicit vertical tracer diffusion step still uses the baseline vertical diffusivity supplied through the parameter set, possibly modified by the machine-learning closure interface. The Richardson-based formulation is therefore best described as an implemented and available closure component whose full online integration into the standard timestep remains a near-term extension.

Overall, the vertical mixing design in OceanJAX reflects a balance between physical realism, numerical stability, and modular extensibility. Implicit vertical solves remove the most restrictive diffusive stability constraints, velocity-consistent masking preserves geometric correctness over irregular bathymetry, and the Richardson-number closure provides a physically meaningful pathway toward stability-dependent turbulent mixing within a differentiable modeling framework.

3.4 Surface Forcing and Boundary Treatment

The treatment of surface forcing and boundary conditions in OceanJAX is designed to keep the physical interpretation of air–sea exchange explicit and numerically unambigu-

ous. Rather than allowing multiple numerical pathways for fluxes to enter the model, the framework separates surface tracer exchange, surface momentum forcing, free-surface kinematics, and interior transport in a mutually consistent way.

For tracers, the most important design principle is that advective exchange across the sea surface is forbidden by construction. The tracer advection routines use the vertical-face mask `mask_w_adv`, whose surface face is always closed. As a consequence, no tracer flux associated with wC is allowed to cross the air–sea interface. All surface exchange of temperature and salinity must therefore be imposed through explicit surface forcing tendencies. This design eliminates the possibility of double counting the same air–sea exchange through both vertical advection and boundary forcing.

Surface tracer forcing is implemented as a top-layer source term. For temperature, the net downward surface heat flux Q_{heat} is converted to a temperature tendency by

$$Q_T = \frac{Q_{\mathrm{heat}}}{\rho_0 c_p \Delta z_1}, \tag{3.26}$$

where c_p is the specific heat capacity of seawater and Δz_1 is the thickness of the uppermost tracer layer. For salinity, OceanJAX uses a virtual salt-flux formulation driven by the net freshwater flux $E - P$:

$$Q_S = \frac{S_{\mathrm{ref}}(E - P)}{\Delta z_1}. \tag{3.27}$$

Under this convention, positive $E - P$ corresponds to net evaporation, that is, freshwater loss from the ocean and therefore an increase in salinity. Both tendencies are applied only at the top tracer layer and are multiplied by the wet surface tracer mask so that no forcing is applied over land.

Surface momentum forcing is handled separately from tracer forcing. Wind stress enters the model through dedicated surface-layer momentum tendencies in the time-stepping module. The zonal and meridional stresses are converted into accelerations applied only to the uppermost velocity layer:

$$\left.\frac{\partial u}{\partial t}\right|_{\tau_x} = \frac{\tau_x}{\rho_0 \Delta z_1}, \qquad \left.\frac{\partial v}{\partial t}\right|_{\tau_y} = \frac{\tau_y}{\rho_0 \Delta z_1}. \tag{3.28}$$

These terms are added to the explicit momentum tendencies before the leapfrog update. Positive τ_x accelerates eastward flow and positive τ_y accelerates northward flow. As with the tracer surface tendencies, the forcing is applied only where the corresponding top-layer velocity points are wet.

The free surface is treated dynamically rather than as a rigid lid. The sea-surface height η evolves from the depth-integrated continuity equation, and after the free-surface update the vertical velocity field is re-diagnosed so that the surface vertical face satisfies the kinematic condition

$$w_{k=0} = \frac{\partial \eta}{\partial t}. \tag{3.29}$$

This treatment is possible because the dynamical vertical-face mask `mask_w` leaves the surface face open wherever the surface tracer cell is wet. In other words, the model distinguishes between a kinematically open surface for dynamics and a tracer-impermeable surface for advective tracer transport.

At the seabed, the treatment is simpler. The bottom vertical face is always closed in the vertical-face masks, which enforces a no-normal-flow condition at the seafloor. For tracer diffusion and tracer advection, this implies zero bottom flux. For the vertical velocity diagnosis, the continuity integration proceeds downward through the water column and terminates at a closed bottom face. In this way, bottom impermeability is embedded directly in the mask structure rather than imposed by separate boundary-condition code branches.

Lateral boundary treatment is determined primarily through the structured grid and face masks. In the zonal direction, the model uses periodic indexing, so east–west differences and fluxes wrap around the domain. In the meridional direction, the northern boundary is treated as a closed wall for meridional transport, implemented by zeroing the corresponding north-face mask. The tracer advection and diffusion operators also apply one-sided or zero-gradient handling at the outer meridional boundary as needed. Thus, the present solver is best understood as a regional spherical patch with periodic zonal closure and wall-type meridional closure, rather than as an open-boundary ocean circulation model.

The data pathway for surface forcing is implemented through the ORAS5 interface. The forcing reader recognizes four two-dimensional fields: net surface heat flux, net freshwater flux, zonal wind stress, and meridional wind stress. It accepts either a single NetCDF file or a list of files, resolves variable-name aliases automatically, and extracts one time slice at a time. The forcing fields are then interpolated horizontally onto the OceanJAX model grid and packaged into a `SurfaceForcing` object used directly by the time stepper.

A particularly important detail in the ORAS5 forcing interface is the treatment of sign conventions and units. The model assumes that heat flux is positive downward into the ocean, freshwater flux is positive for net evaporation $E - P$, and wind stress components are positive eastward and northward. Since ORAS5 freshwater flux may be stored as a mass flux in units of $\mathrm{kg\,m^{-2}\,s^{-1}}$, the loader detects this case and converts it automatically to $\mathrm{m\,s^{-1}}$ by dividing by the freshwater density. Heat flux and wind stress require no further unit conversion. This design ensures that the forcing passed to the numerical core already matches the physical conventions expected by the surface-tendency routines.

In addition to spatially constant forcing, the present framework supports fully time-dependent surface forcing fields. In this formulation, each component of the `SurfaceForcing` object may carry a leading time dimension, for example

$$\texttt{heat_flux} \in R^{n_{\mathrm{steps}} \times N_x \times N_y}.$$

During integration, the `run()` routine advances the model through `jax.lax.scan`, so that at each timestep the forcing sequence is sliced automatically and the single-step solver receives only the instantaneous two-dimensional surface field for that time level. As a result, the heat flux, freshwater flux, and wind stress can vary arbitrarily in time without modifying the structure of the timestep kernel itself. This mechanism allows the model to represent idealized seasonal cycles, synthetic oscillatory forcing, and multi-snapshot reanalysis forcing within the same unified surface-forcing interface.

JAX-Based Implementation

4.1 Functional State Design and Solver Structure

This section focuses on the software structure of OceanJAX as a JAX-based differentiable program. Unlike the previous chapter, which emphasized numerical methods and physical parameterizations, the present discussion concerns how the solver is organized in code so that state evolution, module interaction, automatic differentiation, and later extensions such as machine-learning closures and ensemble parallelism can be handled in a unified way.

A central design choice of OceanJAX is the use of explicit structured state containers. The evolving model variables are grouped into an `OceanState` object, while fixed physical and numerical coefficients are stored in a separate `ModelParams` object. Both are implemented as `equinox.Module` classes. This design gives the solver a clear functional interface: time-dependent fields are carried explicitly from one call to the next, whereas parameters remain logically separate from the evolving state. As a result, the numerical core is expressed as transformations of structured inputs rather than as mutations of hidden global variables.

The `OceanState` container serves not only as a repository for the instantaneous ocean fields, but also as the numerical memory of the solver. In addition to the prognostic variables and diagnostic vertical velocity, it stores the leapfrog history for momentum and free surface, the Adams–Bashforth history for tracer tendencies, and scalar bookkeeping variables such as physical time and step count. This means that a complete numerical state is always available as a single object, which is particularly advantageous in JAX because it allows the full solver trajectory to be represented as repeated transformations of one pytree-like structure.

The state module also provides a small set of constructor functions such as zero-state initialization, resting-state initialization, and initialization from externally supplied arrays. Together with the `apply_masks()` method, these constructors ensure that every state entering the solver already satisfies the expected masking and shape conventions. This reduces hidden assumptions inside the time stepper and makes debugging, testing, and reinitialization more systematic.

On top of these data structures, OceanJAX organizes one model step as a single explicit function,

$$\mathrm{step}(\mathrm{state}, \mathrm{grid}, \mathrm{params}, \mathrm{forcing}, \mathrm{closure}) \rightarrow \mathrm{new_state} \tag{4.1}$$

This function acts as the central solver controller. Rather than embedding physical formulas inside a monolithic implementation, `step()` calls the existing dynamics, tracer, and mixing modules in sequence and assembles the resulting updated state. In this way, the physical modules retain narrow responsibilities: they return tendencies or diagnostic fields only, while all time integration and history rotation are handled in one centralized place.

An important consequence of this design is that optional components are integrated through the same functional interface. Surface forcing is passed explicitly through a `SurfaceForcing` container rather than through external global state. If no forcing is provided, all surface inputs are disabled without altering the solver path. Likewise, the machine-learning interface appears as an optional closure argument. When the closure is absent, the hook is skipped entirely and the numerical path remains identical to the pure-physics solver. When a closure is supplied, it is evaluated once per step and returns a structured output containing tracer-tendency corrections and a vertical-diffusivity scaling factor. This makes the closure pathway local, explicit, and testable.

The multi-step solver is built around the same principle. Rather than iterating in Python, OceanJAX defines a `run()` wrapper that repeatedly applies `step()` through `jax.lax.scan`. The scan carry is the full `OceanState`, and the scanned input is either a time-indexed forcing sequence or a dummy axis when forcing is absent. This approach has several advantages. First, it preserves a single consistent stepping logic for both one-step and long-run integration. Second, it keeps the full time integration loop compatible with JIT compilation. Third, it enables reverse-mode automatic differentiation through the trajectory without requiring a separate adjoint implementation.

The same functional structure also makes batch and ensemble execution natural. The parallel layer does not redefine the solver logic; instead, it reuses `step()` and `run()` through `eqx.filter_vmap` in `batch_step()` and `batch_run()`. In these functions, the leading batch dimension of the state is vectorized, while `grid`, `params`, and the closure object remain shared across the batch. As a result, the same solver can be used for single simulations, batched experiments, and differentiable ensemble calculations without changing the internal time-stepping formulation.

This design extends naturally to the current Phase 1 multi-GPU implementation. In `sharded_ensemble_run()`, the batch dimension is distributed across devices using JAX `NamedSharding`, while `grid` and `params` are replicated on every device. In other words, the present parallelization strategy operates at the level of independent model instances rather than spatial decomposition of a single ocean domain. Although this is not yet full distributed domain decomposition, it demonstrates that the solver structure already supports accelerator-aware scaling because the core numerical interface is expressed cleanly in terms of batched state transformations.

4.2 ORAS5 Data Interface and Regridding

OceanJAX isolates external reanalysis access from the numerical solver through a dedicated ORAS5 data-interface layer. This separation is important for two reasons. First, it prevents dataset-specific naming conventions, coordinate layouts, and missing-value handling from leaking into the solver core. Second, it makes the model easier to test and extend, since reading, interpolation, and state construction are treated as separate tasks rather than being embedded inside the time-stepping code.

The ORAS5 interface follows a two-layer design. The function `read_oras5()` performs pure input/output: it opens a NetCDF file, extracts one time slice, resolves variable names through alias tables, and returns a plain dictionary of NumPy arrays. The function `regrid_to_model()` then takes this dictionary together with an `OceanGrid` object and con-

structs a masked `OceanState` on the model grid. A convenience wrapper, `load_oras5()`, combines these two stages into a single call for routine use. This division is deliberate: it keeps the I/O layer free of JAX dependence while making the interpolation and state-assembly logic explicit.

A key feature of the reader is its support for multiple ORAS5 source layouts. The implementation automatically detects whether the input file uses the regular Copernicus 0.25° longitude–latitude grid or the native NEMO/ORCA curvilinear grid. In the regular case, longitude, latitude, and depth coordinates are read as one-dimensional arrays and sorted into ascending order when necessary. In the curvilinear case, the horizontal coordinates are treated as two-dimensional `nav_lon`/`nav_lat` fields, while the vertical axis is still handled as a one-dimensional depth coordinate. The loader also recognizes that velocity variables may use distinct depth axes such as `depthu` and `depthv`, and it retains these per-field coordinates when they differ from the tracer depth grid.

To improve robustness across product variants, ORAS5 variable names are resolved through priority-ordered alias tables. For example, temperature may be stored under names such as `thetao`, `thetao_oras`, or `votemper`, and the loader accepts these transparently. The same strategy is applied to salinity, velocity, sea-surface height, and the four surface forcing fields. As a result, the data interface is tied less to a specific ORAS5 file naming convention and is more resilient to minor metadata differences across datasets or preprocessing pipelines.

The regridding step maps ORAS5 source fields onto the structured OceanJAX spherical C-grid. Since ORAS5 stores three-dimensional state fields in (N_z, N_y, N_x) order, whereas OceanJAX uses (N_x, N_y, N_z), transposition is applied as part of the conversion. The target horizontal coordinates are taken from `grid.lon_c` and `grid.lat_c`, and the target vertical coordinates are taken from `grid.z_c`. The regridded fields are finally passed to `create_from_arrays()`, which constructs a masked `OceanState` consistent with the model layout.

For regular ORAS5 grids, interpolation is performed through a three-tier NaN-aware strategy. The first tier uses normalized convolution, in which the data field and its validity mask are interpolated separately and then divided, so that coastline-adjacent ocean points are not contaminated by missing-value fill data. The second tier applies nearest-neighbor interpolation only where the linear stencil is entirely invalid. The third tier uses constant fallback values, such as prescribed temperature and salinity fills or zero velocity, to ensure that the target grid is always completely populated. This hierarchy is especially important near coastlines, shallow topography, and the lower boundary of the available source data.

For curvilinear NEMO/ORCA inputs, the horizontal interpolation is handled differently. A Delaunay triangulation is built on a subset of the source grid surrounding the target domain, and `LinearNDInterpolator` is used as the first-stage horizontal interpolant. Missing values are again handled with a normalized-convolution logic, followed by nearest-neighbor fallback on valid source points only. Vertical interpolation is then performed separately from the horizontal step. This two-stage treatment avoids mixing depth and horizontal coordinates in an inconsistent distance metric and keeps the curvilinear regridding both physically interpretable and computationally manageable.

The handling of vertical extrapolation is intentionally field-dependent. For temperature and salinity, values below the deepest valid ORAS5 level are extended by nearest valid source depth through the interpolation fallback logic. For horizontal velocity, how-

ever, the implementation explicitly zeros the field below the deepest available source level. This distinction is motivated physically: extrapolating bottom velocity downward would imply spurious motion beneath the observed or reanalyzed water column, whereas carrying the deepest available tracer value downward is a more benign approximation for hydrographic initialization.

The ORAS5 interface also includes a separate pathway for time-dependent surface forcing. The function `read_oras5_forcing()` reads four two-dimensional fields: net surface heat flux, net freshwater flux, zonal wind stress, and meridional wind stress. These fields are optional; if one is absent, the corresponding entry is returned as `None` rather than raising an error. The function `regrid_forcing()` then interpolates whichever fields are present onto the OceanJAX grid and packages them into a `SurfaceForcing` object, which can be passed directly into the time stepper.

A particularly important part of this forcing pathway is the enforcement of sign conventions and units. The model assumes that heat flux is positive downward into the ocean, freshwater flux is positive for net evaporation ($E - P$), and wind-stress components are positive eastward and northward. The ORAS5 loader checks the freshwater-flux units and, when necessary, converts mass flux in $\mathrm{kg\,m^{-2}\,s^{-1}}$ into the volume-flux form $\mathrm{m\,s^{-1}}$ expected by the tracer surface-forcing routines. Heat flux and wind stress require no analogous conversion in the current implementation. By performing these unit and sign checks inside the data interface, the numerical core can assume that all forcing fields already arrive in solver-ready form.

Finally, the ORAS5 layer is designed to remain loosely coupled to the rest of the model. It depends on the target grid description and on the state-construction interface, but it does not alter the solver logic itself. Initial conditions are delivered as an `OceanState`, and time-dependent boundary inputs are delivered as a `SurfaceForcing` object. This design makes the ORAS5 pathway easy to validate independently and leaves open the possibility of substituting other reanalysis products or observational datasets in the future without restructuring the solver core.

Overall, the ORAS5 interface in OceanJAX should be understood as more than a simple file reader. It is a dedicated data-translation layer that reconciles heterogeneous external datasets with the internal conventions of a differentiable ocean model, including variable naming, coordinate layout, interpolation strategy, unit handling, masking, and state construction.

4.3 Runtime Workflow and NetCDF Output

This subsection describes how OceanJAX is executed as a complete simulation workflow, with emphasis on the connection between initialization, chunked time integration, and persistent NetCDF output. Unlike the previous sections, the focus here is not on the numerical formulation of the timestep itself, but on how the solver is embedded in a runnable experiment pipeline.

In the current implementation, runtime execution is organized by an experiment-level driver script that assembles all required components before integration begins. The workflow starts by constructing the target `OceanGrid` and the corresponding `ModelParams`. An initial `OceanState` is then built according to a user-selected initialization mode, including

idealized rest initialization and ORAS5-based initialization. For ORAS5-driven runs, the script reads one selected time slice, regrids it onto the OceanJAX grid, and then either uses the full reanalysis state or constructs a "cold" state in which temperature and salinity are retained while velocity and sea-surface height are reset to zero. In this way, the runtime layer provides a flexible bridge between data-driven and idealized experiments.

Surface forcing is prepared through a separate runtime path. The driver supports both constant forcing and ORAS5-based forcing read from one or more files. At runtime, forcing fields are assembled chunk by chunk into a `SurfaceForcing` object whose leading dimension matches the number of steps to be executed in the current integration chunk. This design avoids storing an unnecessarily long forcing trajectory in memory and makes the experiment driver compatible with long integrations. In ensemble mode, the same chunked forcing is further broadcast over the ensemble dimension before being passed into the parallel execution layer.

Time integration itself is executed in chunks rather than in one monolithic call. The driver computes the total number of steps from the prescribed number of days and the model time step, and then advances the simulation in blocks of size `CHUNK_SIZE`, subject also to the next scheduled output time. This chunked strategy serves two practical purposes. First, it limits the size of each compiled scan call, which is useful for runtime manageability. Second, it aligns naturally with periodic snapshot writing, since integration can stop exactly at the next save boundary without requiring extra bookkeeping inside the solver kernel. Depending on whether ensemble mode is active, each chunk is advanced either by a JIT-compiled call to `run()` or by the Phase 1 parallel routine `sharded_ensemble_run()`.

A notable aspect of the current workflow is that persistent output is treated as an explicit part of runtime execution rather than as an afterthought. Before time integration begins, the script creates a NetCDF file and writes the initial condition at $t = 0$. During the main integration loop, the updated model state is written again whenever the cumulative step count reaches the next save interval. Thus, the output file is constructed incrementally as the simulation proceeds, rather than being produced only at the very end. This approach reduces the risk of losing the full trajectory in the event of interruption and makes intermediate results immediately available for inspection.

The current NetCDF layout is now sufficiently rich to represent both single-member and ensemble simulations. In all cases, the file defines the dimensions `time`, `x`, `y`, `z`, and `zw`, where `zw` corresponds to the vertical face coordinate used by the diagnosed vertical velocity. When ensemble mode is enabled, an additional `member` dimension is created. The main prognostic and diagnostic fields written to file are temperature `T`, salinity `S`, free-surface height `eta`, horizontal velocity components `u` and `v`, and vertical velocity `w`. Their dimensions are expanded consistently in ensemble mode, so that the output file preserves the full member-wise model trajectory.

The file also stores coordinate variables and experiment metadata. In particular, the coordinates `x`, `y`, `z`, and `zw` are written from the target model grid, while global attributes record the domain bounds, time step, integration length, prescribed surface forcing values, initialization mode, and ensemble size. This metadata layer is important because it makes each output file self-describing and therefore suitable for downstream validation, comparison, or visualization without requiring the original runtime configuration to be reconstructed manually.

From a workflow perspective, the NetCDF writer is deliberately kept outside the numerical timestep kernel. The solver itself returns updated state objects, while the experiment driver decides when those states should be serialized. This separation preserves the numerical purity of the stepping functions and avoids coupling solver logic to file-format operations. At the same time, the output interface remains close enough to the runtime loop that save frequency, ensemble shape, and metadata can be controlled directly at the experiment level.

The runtime script also integrates lightweight online diagnostics into the same loop. After each chunk, it prints the current simulated day, step count, basic field ranges, and wall-clock time, and it explicitly checks for non-finite values in the evolving state. In ensemble mode, these diagnostics are reported as ensemble mean ranges together with ensemble spread. This means that the runtime workflow already combines numerical integration, persistent storage, and basic health monitoring within a single reproducible execution path.

4.4 ML and Parallel Extensions

OceanJAX extends the core solver in two directions: a machine-learning closure interface and a Phase 1 ensemble-parallel execution layer. Both extensions are designed to remain compatible with the existing solver structure, so that they reuse the same timestep kernel rather than introducing separate code paths.

Machine-learning closure interface. The ML extension is implemented through a closure hook defined in `closure.py`. Any closure returns a `ClosureOutput` containing three quantities: an additive temperature-tendency correction, an additive salinity-tendency correction, and a multiplicative scaling applied to the background vertical diffusivity. The closure is evaluated once per timestep after the explicit tracer tendencies are formed and before the implicit vertical mixing step. This placement allows learned corrections to modify tracer evolution and effective vertical mixing while preserving the existing numerical structure of the solver.

The interface is intentionally restricted in its current form. It targets tracer-side parameterization only; momentum corrections are not yet included because they would interact more delicately with the leapfrog and Asselin–Robert time integration used for the dynamical core. Since closures are implemented as `equinox.Module` objects, their parameters remain compatible with `jax.grad`, `jax.jit`, and `eqx.filter_vmap`, which makes the full integration trajectory differentiable with respect to closure parameters.

Phase 1 ensemble parallelism. The current parallel extension operates over an ensemble or batch dimension rather than over a spatial domain decomposition. The functions `batch_step()` and `batch_run()` use `eqx.filter_vmap` to vectorize the single-member `step()` and `run()` routines over a leading batch axis, while sharing the same grid, parameters, and closure across all members. This makes it possible to run multiple independent model realizations simultaneously without altering the underlying solver logic.

For larger ensembles, `sharded_ensemble_run()` distributes the batch axis across multiple devices using JAX `NamedSharding`. In this configuration, state and forcing arrays are

sharded along the leading dimension, while the grid, parameters, and closure are replicated on all devices. This is therefore a Phase 1 parallel strategy aimed at independent-instance execution rather than single-domain multi-GPU ocean simulation. Even so, it provides an immediate pathway for ensemble experiments, uncertainty studies, and future closure-training workflows.

Preliminary Verification and Discussion

5.1 Numerical Consistency and Stability Checks

5.1.1 Experiment 5.1.1: Resting Uniform Ocean Preservation Test

Objective. This experiment verifies that, under idealized conditions with no external forcing and no initial motion, the momentum and free-surface operators do not generate spurious flow. It further checks that the volume-integrated tracer drift remains within the expected range of single-precision floating-point roundoff error and accumulates linearly rather than exponentially, indicating the absence of numerical instability.

Table 5.1: Configuration of Experiment 5.1.1.

Parameter	Value
Domain	40°W–5°W, 15°S–15°N
Grid	$20 \times 15 \times 6$, flat-bottom bathymetry
Vertical levels	25, 75, 150, 250, 375, 500 m
Time step Δt	300 s
Integration length	30 days (8640 steps)
Initial temperature / salinity	Uniform 15.0°C / 35.0 psu
Initial u, v, η	0
Surface forcing	None
Output interval	Once per day (every 288 steps)

The experiment passed all diagnostics. Velocity, free surface, and kinetic energy remained exactly zero throughout the 30-day integration, demonstrating that the discrete momentum and pressure-gradient operators did not generate spurious motion in a perfectly resting ocean. Meanwhile, the volume-integrated tracer quantities exhibited only a very small linear drift, with rates of 5.17×10^{-6} day^{-1} for temperature and 1.48×10^{-6} day^{-1} for salinity. This behavior is consistent with accumulated single-precision roundoff error introduced by the implicit vertical mixing solver, specifically the Thomas algorithm, which contributes an error of approximately 4.8×10^{-7} per time step (roughly four times machine epsilon in `float32`). Over 288 steps per day, this error accumulates linearly, and the fitted coefficients of determination ($R^2 > 0.9999$) confirm that the drift is truncation-like rather than unstable growth.

Table 5.2: Results of Experiment 5.1.1.

Metric	Result	Criterion	Outcome
Relative volume drift of T (maximum over 30 days)	1.55×10^{-4}	$< 5 \times 10^{-4}$	PASS
Relative volume drift of S (maximum over 30 days)	4.43×10^{-5}	$< 5 \times 10^{-4}$	PASS
Linearity of T drift (R^2)	0.999999	> 0.999	PASS
Linearity of S drift (R^2)	0.999992	> 0.999	PASS
$\max\lvert u\rvert$, $\max\lvert v\rvert$	0.000 m s^{-1} (exact)	$< 10^{-10}$	PASS
$\max\lvert\eta\rvert$	0.000 m (exact)	$< 10^{-10}$	PASS
Kinetic energy (KE)	0.000 J (exact)	$< 10^{-6}$	PASS
Non-finite values	None	—	PASS

5.1.2 Experiment 5.1.2: Stratified Rest Test over Real Topography

Objective. This experiment evaluates whether the pressure-gradient and continuity operators remain mutually consistent under realistic ORAS5-derived bathymetry, including irregular bottom topography and land–sea masking. Starting from a horizontally uniform but vertically stratified initial state, the test examines whether the model generates any spurious circulation at mask boundaries or partial-bottom cells. Compared with the flat-bottom resting-ocean case, this is a more stringent verification, because irregular topography and coastline geometry can introduce additional truncation errors in the discrete pressure-gradient calculation and may drive artificial recirculation if the operators are not numerically consistent.

Table 5.3: Configuration of Experiment 5.1.2.

Parameter	Value
Domain	40°W–5°W, 15°S–15°N
Grid	$20 \times 15 \times 6$
Bathymetry	Real bottom depth derived from the ORAS5 wet mask (lower interface of the deepest wet cell)
Initial temperature / salinity	Domain-mean ORAS5 vertical profiles, horizontally broadcast over the full domain
Temperature profile	26.6 / 21.6 / 15.4 / 12.1 / 9.6 / 7.9 °C (surface to bottom)
Salinity profile	36.0 / 36.2 / 35.7 / 35.2 / 34.9 / 34.7 psu (surface to bottom)
Initial u, v, η	0
Surface forcing	None
Time step Δt	300 s
Integration length	10 days (2880 steps)
Wet horizontal cells	300 / 300 (the selected tropical Atlantic domain is entirely ocean-covered at the horizontal grid scale)

Table 5.4: Results of Experiment 5.1.2.

Metric	Result	Criterion	Outcome
$\max \lvert u \rvert$	0.000 m s^{-1} (exact)	$< 10^{-3}$	PASS
$\max \lvert v \rvert$	0.000 m s^{-1} (exact)	$< 10^{-3}$	PASS
$\max \lvert \eta \rvert$	0.000 m (exact)	$< 10^{-3}$	PASS
Kinetic energy (KE)	0.000 J (exact)	$< 10^{6}$	PASS
Non-finite values	None	—	PASS
Maximum pointwise T drift (10 days)	1.65×10^{-2} °C	—	Linear accumulation

The experiment passed all diagnostics. Velocity, free surface, and kinetic energy remained exactly zero throughout the 10-day integration, demonstrating that the pressure-gradient and continuity operators did not generate any spurious circulation under realistic bathymetry and masking conditions. The maximum pointwise temperature drift over 10 days was 1.65×10^{-2} °C, corresponding to an approximately linear rate of 1.65×10^{-3} °C day^{-1}. This magnitude is consistent with the `float32` truncation error previously identified in Experiment 5.1.1 as originating from the Thomas algorithm used in the implicit vertical mixing solver. A notable feature of this setup is that all 300 horizontal grid cells in the selected tropical Atlantic domain remain wet in the $20 \times 15 \times 6$ configuration, so the bathymetric effect is expressed primarily through vertically varying bottom cell

thickness rather than horizontal coastline truncation. The absence of any artificial motion indicates that the model remains numerically balanced even in the presence of realistic partial-bottom structure.

5.1.3 Experiment 5.1.3: Zero-Side-Effect Verification of `NullClosure`

Objective. This experiment verifies that the machine-learning closure interface produces exactly the same numerical solution when supplied with `NullClosure()` as when no closure is provided (`closure=None`). The goal is to confirm that the ML hook is numerically transparent when inactive, meaning that it introduces no side effects into the physical computation. This serves as a direct verification of the zero-overhead design principle of the ML interface.

Table 5.5: Configuration of Experiment 5.1.3.

Parameter	Value
Initial condition	`oras5_cold` (T/S initialized from ORAS5, with $u = v = \eta = 0$)
Surface forcing	Full forcing (surface heat flux, freshwater flux, zonal and meridional wind stress)
Time step Δt	300 s
Integration length	1 day (288 steps)
Run A	`closure=None`
Run B	`closure=NullClosure()`
Comparison method	Pointwise maximum absolute difference over all prognostic fields

Table 5.6: Results of Experiment 5.1.3.

Field	$\max \lvert A - B \rvert$	Bit-identical	Outcome
T	0.000e+00	YES	PASS
S	0.000e+00	YES	PASS
η	0.000e+00	YES	PASS
u	0.000e+00	YES	PASS
v	0.000e+00	YES	PASS

The experiment passed all diagnostics. After a 1-day integration (288 time steps), all five prognostic fields remained strictly bit-identical between the reference run with `closure=None` and the test run with `closure=NullClosure()`. The pointwise maximum absolute difference was exactly zero for temperature, salinity, free surface, and both horizontal velocity components. This result demonstrates that when `NullClosure` returns zero tendency corrections and a unit `kappa_v_scale`, the ML hook remains completely transparent to the numerical solution and introduces no floating-point branching differences or unintended perturbations along the computational pathway.

5.1.4 Experiment 5.1.4: Time-Step Stability Scan under ORAS5-Initialized Forcing

Objective. This experiment systematically scans the numerical stability boundary of the model under different time-step sizes using the `oras5_cold` initial condition and full surface forcing, rather than an idealized resting ocean. A stability limit derived from a realistically structured initial state is more representative of actual production runs and therefore provides a more conservative and convincing upper bound for the admissible time step.

Table 5.7: Configuration of Experiment 5.1.4.

Parameter	Value
Initial condition	`oras5_cold` (T/S initialized from ORAS5, with $u = v = \eta = 0$)
Surface forcing	Full forcing (four ORAS5 forcing files)
Domain	40°W–5°W, 15°S–15°N, 500 m depth
Grid	$20 \times 15 \times 6$, $\Delta x_{\min} = 188.8$ km
Barotropic wave speed	$c = \sqrt{gH} = \sqrt{9.81 \times 500} \approx 70.0$ m s^{-1}
Scanned time steps	100, 300, 600, 900, 1200 s
Integration length	7 days for each run
Stability criteria	No NaN/Inf; $T \in (0, 50)$ °C; S (wet cells only) $\in (30, 42)$ psu

Table 5.8: Results of Experiment 5.1.4.

Δt [s]	Theoretical CFL	T range [°C]	S range [psu]	max $\lvert\eta\rvert$ [m]	Outcome
100	0.037	[6.77, 28.70]	[34.59, 37.26]	0.196	STABLE
300	0.111	[6.77, 28.68]	[34.58, 37.24]	0.188	STABLE
600	0.223	[6.76, 28.66]	[34.54, 37.25]	0.185	STABLE
900	0.334	[6.75, 28.62]	[34.45, 37.26]	0.182	STABLE
1200	0.445	NaN	NaN	NaN	UNSTABLE

The experiment passed its intended verification objective by clearly identifying a finite numerical stability boundary. Four out of the five tested time steps remained stable over the 7-day integrations, while the run with $\Delta t = 1200$ s became unstable and produced NaN values. The stability limit therefore lies between CFL $= 0.334$ ($\Delta t = 900$ s) and CFL $= 0.445$ ($\Delta t = 1200$ s). The production configuration adopted in the present study, $\Delta t = 300$ s, corresponds to CFL ≈ 0.11, which places it comfortably within the stable regime and provides approximately a factor-of-three safety margin relative to the empirical instability threshold. The unstable behavior at $\Delta t = 1200$ s is consistent with violation of the barotropic gravity-wave CFL constraint in the leapfrog time-stepping scheme, where uncontrolled free-surface oscillations subsequently contaminate the tracer fields. Among the stable runs, the ranges of temperature, salinity, and free-surface height vary only slightly with increasing Δt, indicating that numerical dissipation remains negligible within the regime CFL < 0.35.

5.1.5 Experiment 5.1.5: Vertical Mixing Parameterization Test

Objective. This experiment verifies the physical correctness of the implicit vertical mixing solver and the `KappaScaleClosure.kappa_v_scale` interface. The expected behavior is that a larger vertical diffusivity produces stronger smoothing of the temperature profile, while the limiting case $\kappa_v = 0$ preserves the initial temperature structure exactly. The test therefore examines both the monotonic physical response of the parameterization and the numerical transparency of the diffusivity-scaling closure interface.

Table 5.9: Configuration of Experiment 5.1.5.

Parameter	Value
Grid	$5 \times 5 \times 6$, flat-bottom bathymetry, 500 m depth, tropical Atlantic domain
Initial condition	Step-like thermocline: $T = 22°\mathrm{C}$ in the upper three layers (≤ 150 m) and $T = 10°\mathrm{C}$ in the lower three layers (> 150 m); uniform $S = 35$ psu
Dynamics	No initial motion, no surface forcing
Time step Δt	300 s
Integration length	30 days
Diagnostic quantity	Vertical temperature-profile smoothing measured by the final-to-initial standard-deviation ratio (`std_ratio`) and maximum temperature change

Table 5.10: Test groups in Experiment 5.1.5.

Group	Effective κ_v	Implementation
A	1×10^{-4} m^2 s^{-1}	`closure=None`
B	1×10^{-2} m^2 s^{-1}	`KappaScaleClosure(scale=100)`
C	0 m^2 s^{-1}	`KappaScaleClosure(scale=0)`

Table 5.11: Results of Experiment 5.1.5.

Group	`std_ratio` (final / initial)	Maximum ΔT	Physical interpretation
A	0.9821	0.337°C	Very weak mixing over 30 days ($\tau \approx 730$ days), consistent with expectation
B	0.3701	5.649°C	Strong smoothing of the thermocline ($\tau \approx 7$ days), with a 63% reduction in profile standard deviation
C	1.0000	0.000°C	Exact preservation of the temperature profile for $\kappa_v = 0$

The experiment passed all diagnostics, with all six validation checks satisfied and no NaN values encountered. The results exhibit the correct monotonic ordering of mixing

strength, namely $\texttt{std_ratio}_C > \texttt{std_ratio}_A > \texttt{std_ratio}_B$. Group A, using the default background diffusivity, showed only weak smoothing over 30 days, which is consistent with the long theoretical diffusion timescale. Group B, with diffusivity increased by a factor of 100 through `KappaScaleClosure`, produced strong thermocline erosion and substantial profile homogenization. Group C, with $\kappa_v = 0$, preserved the initial temperature profile exactly to machine precision. These results confirm both the physical correctness of the implicit vertical mixing formulation and the intended functionality of the diffusivity-scaling closure interface.

5.2 ORAS5-Initialized Integration Results

5.2.1 Experiment 5.2.1: `oras5_cold` with Full Surface Forcing

Objective. This experiment serves as the baseline ORAS5-initialized integration and evaluates whether the model produces physically consistent short-term evolution under prescribed ORAS5 surface forcing. The purpose is to verify that the coupled response of temperature, salinity, and free surface remains numerically stable and dynamically plausible over a 30-day integration.

Table 5.12: Configuration of Experiment 5.2.1.

Parameter	Value
Domain	Tropical Atlantic, 40°W–5°W, 15°S–15°N
Grid	$20 \times 15 \times 10$
Initial condition	`oras5_cold` (T/S initialized from ORAS5, with $u = v = \eta = 0$)
Surface forcing	Full ORAS5 surface forcing
Time step Δt	300 s
Integration length	30 days

Table 5.13: Results of Experiment 5.2.1.

Metric	Result	Outcome
Sea-surface temperature (SST)	26.6°C → 23.6°C	
Net SST change	−3.0°C	PASS
Salinity range	34.47–37.26 psu	PASS
Maximum free-surface amplitude, $\max\lvert\eta\rvert$	0.60 m	PASS
Non-finite values	None	PASS

The experiment passed all diagnostics. Over the 30-day integration, the model remained numerically stable and produced bounded, physically plausible evolution from the ORAS5-derived thermal and haline initial state. The sea-surface temperature decreased from 26.6°C to 23.6°C, while salinity remained within the physically reasonable range of 34.47–37.26 psu and the maximum free-surface amplitude reached 0.60 m without generating any NaN values. The SST cooling is consistent with the prescribed January

surface forcing and the model's internal vertical redistribution, indicating that the ORAS5-initialized, fully forced configuration can support stable short-term integration.

5.2.2 Experiment 5.2.2: `oras5_cold` without Surface Forcing

Objective. This experiment provides a control integration for comparison with Experiment 5.2.1. By removing all surface forcing while retaining the same ORAS5-derived thermohaline initial condition, the test isolates the portion of the short-term evolution associated with internal adjustment, vertical mixing, and dynamical redistribution.

Table 5.14: Configuration of Experiment 5.2.2.

Parameter	Value
Domain	Tropical Atlantic, 40°W–5°W, 15°S–15°N
Grid	$20 \times 15 \times 10$
Initial condition	`oras5_cold` (T/S initialized from ORAS5, with $u = v = \eta = 0$)
Surface forcing	None
Time step Δt	300 s
Integration length	30 days

Table 5.15: Results of Experiment 5.2.2.

Metric	Result	Outcome
Sea-surface temperature (SST)	26.6°C → 24.2°C	
Net SST change	−2.4°C	PASS
Salinity range	34.47–37.26 psu	PASS
Maximum free-surface amplitude, $\max\lvert\eta\rvert$	0.14 m	PASS
Non-finite values	None	PASS

The experiment passed all diagnostics. Even in the absence of surface forcing, the model exhibited substantial SST cooling, from 26.6°C to 24.2°C, while salinity remained within the same physically bounded range as in the fully forced case. The maximum free-surface amplitude was considerably smaller than in Experiment 5.2.1, reaching only 0.14 m, which indicates a weaker dynamic response without external forcing. Comparison with the fully forced baseline suggests an additional cooling contribution of approximately 0.6°C associated with the applied surface forcing over 30 days, whereas the remaining cooling appears to arise primarily from internal adjustment, vertical mixing, and redistribution of the ORAS5-initialized stratification. This result therefore confirms that a substantial portion of the short-term evolution is intrinsic to the initialized state rather than externally imposed.

5.2.3 Experiment 5.2.3: `oras5_full` without Surface Forcing

Objective. This experiment evaluates whether the model remains stable when initialized from the full ORAS5 state, including velocity and free-surface structure, rather than

from a dynamically quiescent thermohaline field. The purpose is to verify that the model can ingest and evolve the complete ORAS5 initial condition without salinity blow-up or free-surface instability.

Table 5.16: Configuration of Experiment 5.2.3.

Parameter	Value
Domain	Tropical Atlantic, 40°W–5°W, 15°S–15°N
Grid	$20 \times 15 \times 10$
Initial condition	`oras5_full` (full ORAS5 state, including velocity and free surface; maximum initial $\lvert u \rvert \approx 0.73$ m s^{-1})
Surface forcing	None
Time step Δt	300 s
Integration length	30 days

Table 5.17: Results of Experiment 5.2.3.

Metric	Result	Outcome
Salinity range	34.08–37.25 psu	PASS
Maximum free-surface amplitude, $\max \lvert \eta \rvert$	0.77 m	PASS
Non-finite values	None	PASS
Overall integration stability	Stable over 30 days	PASS

The experiment passed all diagnostics. Starting from the full ORAS5 state, including dynamically active velocity and free-surface structure, the model remained stable throughout the 30-day integration and produced no NaN values. Salinity stayed within the physically reasonable range of 34.08–37.25 psu, and the maximum free-surface amplitude reached 0.77 m. These results show that the model can robustly evolve a dynamically non-resting ORAS5 initial condition without triggering salinity instability or numerical blow-up. In this sense, the experiment provides strong evidence that the full-state initialization pathway is numerically stable. Because surface forcing is disabled in this case, the result should be interpreted primarily as a verification of stable full-state initialization rather than a direct standalone test of the freshwater-flux forcing term.

Table 5.18: Summary of Experiments 5.2.1–5.2.3.

Case	Surface forcing	SST change	Salinity range	max $\lvert\eta\rvert$
Experiment 5.2.1: `oras5_cold` + full forcing	Full ORAS5 forcing	$-3.0°$C	34.47–37.26 psu	0.60 m
Experiment 5.2.2: `oras5_cold` + no forcing	None	$-2.4°$C	34.47–37.26 psu	0.14 m
Experiment 5.2.3: `oras5_full` + no forcing	None	—	34.08–37.25 psu	0.77 m

5.2.4 Supplementary Experiment: 180-Day Atlantic Basin Integration

Objective. This supplementary experiment extends the integration domain from the tropical Atlantic test region to a much larger Atlantic basin configuration and lengthens the simulation period to 180 days. Its purpose is to assess whether the model remains numerically stable under basin-scale geometry and sustained forcing, and whether it can produce qualitatively reasonable large-scale adjustments, including basin-wide sea-surface height gradients and broad thermohaline redistribution. Because the forcing is constructed by repeatedly applying January ORAS5 surface fields and the model still uses simplified physics and boundary treatment, this experiment should be interpreted as a large-scale demonstration rather than a strict validation against reality.

Table 5.19: Configuration of the supplementary 180-day Atlantic basin experiment.

Parameter	Value
Domain	Atlantic basin, 80°W–20°E, 50°S–60°N
Grid	$50 \times 40 \times 10$
Maximum depth	1000 m
Initial condition	`oras5_cold`
Surface forcing	Full ORAS5 surface forcing
Time step Δt	300 s
Integration length	180 days

Table 5.20: Results of the supplementary 180-day Atlantic basin experiment.

Metric	Initial	Final (Day 180)	Change / interpretation
Mean SST	17.45°C	12.77°C	−4.68°C
Maximum temperature (tropics)	28.4°C	17.3°C	Approx. −11°C
Minimum temperature	−1.53°C	3.79°C	Polar cold water weakened by mixing
Minimum salinity	6.12 psu	33.89 psu	Extremely fresh initial water mass mixed away rapidly
Minimum free surface, $\eta_{\min}$	−1.15 m	−11.96 m	Strong deepening of basin-scale cyclonic signal
Maximum free surface, $\eta_{\max}$	+2.76 m	+6.30 m	Strengthening of basin-scale anticyclonic signal

The experiment remained numerically stable throughout the full 180-day basin-scale integration and therefore demonstrates that the model can be run far beyond the short regional verification cases without numerical blow-up. Several large-scale tendencies emerged clearly. First, the free-surface signal amplified continuously, with the basin-wide range of η increasing from approximately 4 m initially to more than 18 m by Day 180. This indicates sustained adjustment of the large-scale circulation under wind-stress forcing and is qualitatively consistent with the formation of subtropical and subpolar gyre-like structures. Second, sea-surface temperature underwent persistent cooling, with the basin-mean SST decreasing by 4.68°C and tropical maximum temperature dropping by about 11°C. This behavior is consistent with the repeated application of wintertime forcing and the absence of a seasonal forcing cycle. Third, the extremely low initial salinity minimum rapidly relaxed toward a more physical range within the first few weeks, suggesting that the very fresh initial values near the ORAS5 boundary were numerical or interpolation artifacts rather than dynamically maintained water masses.

At the same time, the magnitude of the final free-surface anomaly is substantially larger than would be expected in the real Atlantic Ocean, and the long-term thermal drift is also exaggerated. These features indicate that the experiment should not be interpreted as a realistic hindcast. Instead, it is better understood as a qualitative stress test of the model under large-domain, long-duration integration. The inflated basin-scale response likely reflects the combined effects of repeated single-month forcing, the absence of seasonal cycling, simplified mixing physics, coarse resolution, and the present closed-boundary treatment. Nevertheless, the successful completion of the 180-day run demonstrates that the current OceanJAX framework is already capable of stable basin-scale integrations and of generating dynamically organized large-scale adjustment patterns.

5.3 Discussion of Current Model Behavior

The present experiments indicate that the current OceanJAX framework already exhibits a coherent and physically interpretable model behavior, although it should still be regarded

as a preliminary verification system rather than a fully validated ocean prediction model. The resting-state and stratified-rest tests show that the pressure-gradient, continuity, and masking operators are mutually consistent, since no spurious circulation is generated even under realistic bathymetry, while the very small tracer drift is attributable to linear accumulation of `float32` roundoff error in the implicit vertical mixing solver rather than numerical instability. The time-step scan further identifies a practical stability boundary between approximately CFL $= 0.33$ and 0.45, confirming that the production choice of $\Delta t = 300$ s lies well within the stable regime. The vertical mixing experiment demonstrates the correct monotonic response to diffusivity changes, with stronger κ_v producing stronger thermocline erosion and $\kappa_v = 0$ preserving the initial profile exactly, which also confirms the correctness of the `kappa_v_scale` closure interface. In the 30-day ORAS5-initialized integrations, the model remains stable under both cold-start and full-state initial conditions, and produces bounded, physically plausible evolution of temperature, salinity, and free surface; comparison between forced and unforced cold-start runs suggests that short-term cooling is jointly controlled by prescribed surface forcing and internal adjustment of the initial stratification. The 180-day Atlantic basin experiment further shows that the framework can sustain long, basin-scale integrations and develop dynamically organized large-scale adjustment patterns, but the excessive amplitude of the resulting free-surface signal also makes clear that the current configuration still suffers from important limitations, including repeated single-month forcing, coarse resolution, simplified physics, and the absence of more realistic boundary treatment. Finally, the bit-identical agreement between `NullClosure()` and `closure=None` confirms that the ML interface is numerically transparent when inactive.

Limitations and Future Work

Although the present OceanJAX framework has demonstrated stable and physically interpretable behavior across a range of idealized, regional, and basin-scale experiments, several important limitations remain. First, the current simulations are still conducted at relatively coarse spatial resolution, which limits the representation of mesoscale structures, boundary currents, and sharp thermohaline fronts. Second, the forcing strategy used in the longer integrations is simplified, and repeated application of a limited set of ORAS5 surface fields cannot fully represent realistic seasonal or interannual variability. Third, the present boundary treatment and simplified physics, including reduced-form vertical mixing and the absence of more complete subgrid-scale parameterizations, likely contribute to the exaggerated large-scale free-surface response seen in the 180-day Atlantic experiment. Fourth, the current evaluation remains primarily a consistency and stability verification rather than a full quantitative validation against observations or reanalysis targets. Finally, although the machine-learning interface has been shown to be numerically transparent and physically controllable, no learned closure has yet been trained or deployed in prognostic simulations.

Future work will therefore focus on improving both physical realism and predictive credibility. Planned developments include higher-resolution and convergence experiments, the introduction of temporally varying seasonal forcing, improved open-boundary or relaxation schemes, and more complete physical parameterizations such as enhanced vertical mixing and nonlinear momentum processes. A systematic validation framework will also be developed using ORAS5 and other observationally constrained datasets, with diagnostics based on temperature, salinity, sea-surface height, and circulation statistics. In parallel, the existing ML interface will be extended toward learned tracer-tendency and diffusivity closures trained on high-resolution reference data, while the newly implemented multi-GPU capability will support longer and larger-domain integrations needed for future basin-scale and multi-year experiments.

Conclusion

This project developed *OceanJAX*, a differentiable three-dimensional ocean model implemented in JAX, with the goal of combining physically based ocean simulation, modern automatic differentiation, and extensibility toward machine-learning-assisted parameterizations. The model is formulated on an Arakawa C-grid in spherical coordinates, supports variable horizontal resolution and realistic bathymetry, and includes a hydrostatic Boussinesq dynamical core, tracer transport, free-surface evolution, implicit vertical mixing, ORAS5-based initialization and forcing, and a modular interface for future ML closures and parallel extensions. In this sense, the present work establishes a complete end-to-end framework that links grid generation, state representation, numerical operators, physical parameterizations, runtime integration, and NetCDF-based output into a unified JAX-based ocean modeling system.

A series of preliminary verification experiments shows that the current framework is numerically stable, internally consistent, and physically interpretable. Resting-ocean and stratified-rest tests demonstrate that the pressure-gradient, continuity, and masking operators do not generate spurious circulation, while the small tracer drift observed in these experiments is consistent with linear accumulation of single-precision roundoff error rather than numerical instability. The time-step stability scan identifies a practical CFL stability boundary and confirms that the production time step used in the main experiments lies comfortably within the stable regime. Vertical mixing tests further show that the implicit diffusion scheme responds monotonically and correctly to changes in diffusivity, and that the `kappa_v_scale` ML interface can modulate physical mixing strength in a controlled way. ORAS5-initialized integrations over both regional and basin-scale domains remain stable and produce dynamically organized, physically plausible short-term evolution, demonstrating that the model can already support meaningful ocean adjustment experiments beyond idealized test cases.

Bibliography

[1] Arakawa, A., and Lamb, V. R. (1977). Computational design of the basic dynamical processes of the UCLA general circulation model. *Methods in Computational Physics*, **17**, 173–265.

[2] Griffies, S. M., and Adcroft, A. J. (2008). Formulating the equations of ocean models. In M. W. Hecht and H. Hasumi (eds.), *Ocean Modeling in an Eddying Regime*, Geophysical Monograph Series, Vol. 177, pp. 281–317. Washington, DC: American Geophysical Union.

[3] Asselin, R. (1972). Frequency filter for time integrations. *Monthly Weather Review*, **100**(6), 487–490.

[4] Pacanowski, R. C., and Philander, S. G. H. (1981). Parameterization of vertical mixing in numerical models of tropical oceans. *Journal of Physical Oceanography*, **11**(11), 1443–1451.

[5] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). *JAX: Composable Transformations of Python+NumPy Programs*. Available at: `https://github.com/jax-ml/jax`

[6] Large, W. G., McWilliams, J. C., and Doney, S. C. (1994). Oceanic vertical mixing: A review and a model with a nonlocal boundary layer parameterization. *Reviews of Geophysics*, **32**(4), 363–403.

Appendix

.1 Default Model Parameters

This appendix summarizes the default numerical and physical parameters used in the present OceanJAX configuration. Unless otherwise stated, these values correspond to the baseline setup adopted in the main verification experiments. For experiments that intentionally modify individual settings, such as time step, diffusivity, or forcing configuration, the altered values are described explicitly in the corresponding sections of the main text.

Table 1 summarizes the main parameters currently used in `ModelParams`, including their default values, units, and physical meanings.

Table 1: Reference table for `ModelParams`.

Parameter	**Default**	**Unit**	**Description**
`dt`	900.0	s	Time step (override to 300 s)
`rho0`	1025.0	kg/m^3	Reference density (Boussinesq)
`g`	9.81	m/s^2	Gravitational acceleration
`alpha_T`	2×10^{-4}	1/K	Thermal expansion coefficient
`beta_S`	7.4×10^{-4}	1/psu	Haline contraction coefficient
`T_ref`	10.0	°C	EOS reference temperature
`S_ref`	35.0	psu	EOS reference salinity
`nu_h`	200.0	m^2/s	Horizontal viscosity
`nu_v`	1×10^{-4}	m^2/s	Vertical viscosity
`kappa_h`	100.0	m^2/s	Horizontal tracer diffusivity
`kappa_v`	1×10^{-5}	m^2/s	Vertical tracer diffusivity
`asselin_coeff`	0.1	–	Asselin–Robert filter coefficient
`ab3_coeffs`	(23/12, −16/12, 5/12)	–	Adams–Bashforth 3 coefficients

In the present study, the baseline configuration was chosen to prioritize numerical robustness, interpretability, and modular extensibility rather than maximum physical complexity. As a result, the parameter set listed above should be understood as the default research configuration of the current OceanJAX framework. Individual experiments in the main text intentionally vary selected parameters, such as the time step, vertical diffusivity, surface forcing, or initialization pathway, in order to isolate specific numerical and physical behaviors.

.2 Source Code

All source code is provided in the following github repository.
https://github.com/Ethreall418/EAEE9280$_{P}ersonal_{R}esearch_{P}roject$